

Technical Reflection Report: TechMoveGLMS Modernization

Date: June 20, 2026

Project: TechMove Logistics Management System (GLMS)

1. Introduction

This report reflects on the architectural transition of the TechMoveGLMS from a monolithic MVC application to a modern, decoupled system using a Web API, comprehensive unit testing, and containerization.

2. Architectural Transition (DevOps)

The primary shift involved decoupling the data access layer from the user interface.

- **Decoupling:** By introducing a Web API (`TechMoveGLMS.API`), we isolated the business logic and database interactions. The MVC application was refactored to act as a “thin client,” communicating solely via `IApiService` and `HttpClient` .
- **Scalability:** This architecture allows the backend and frontend to scale independently. The API can now support multiple clients, such as mobile apps or third-party integrations, without modifying the core logic.

3. Quality Assurance (Testing)

To ensure system reliability, an xUnit test project was implemented.

- **Service Testing:** Unit tests were created for critical services like `CurrencyService` (external API integration) and `FileValidationService` (security and business rules).
- **Controller Testing:** Using `InMemoryDatabase` , we simulated API requests to verify that business constraints—such as blocking service requests for expired contracts—are strictly enforced at the API level.
- **Impact:** Automated testing significantly reduces the risk of regressions during future updates.

4. Containerization (Docker)

The solution was containerized using Docker and orchestrated with Docker Compose.

- **Environment Consistency:** Docker ensures that the application runs identically on a developer's machine, in a testing environment, and in production.
- **Multi-Container Setup:**
 - `glms-backend-api` : Handles logic and data.
 - `glms-frontend-web` : Serves the user interface.
 - **Data Persistence:** A shared Docker volume ensures the SQLite database persists across container restarts and is accessible by both services.

5. Challenges and Solutions

- **Currency Conversion:** Implementing real-time conversion required a mix of server-side logic (for data integrity) and client-side JavaScript (for user experience).
- **Docker Build Issues:** Addressed environment-specific build errors by implementing a `.dockerignore` file to optimize the build context and prevent metadata corruption.

6. Conclusion

The modernization of TechMoveGLMS has resulted in a more robust, testable, and deployable application. The transition to a service-oriented architecture, backed by containerization, aligns the project with industry-standard DevOps practices.